

康托展开与求逆的一点总结

数据量为n时，抛开阶乘的高精度计算，康托展开可以借助树状数组（BIT/Fenwick树）在 $O(n\log n)$ 复杂度下实现：

```
#define N 10010
int a[N], c[N] = {0}, n; long long factor[N];
long long cantor() {
    long long ans;
    for (int i=n-1; i>=0; --i) {
        int s = 0;
        for (int x=a[i]; x>0; x -= x&(-x)) s += c[x];
        ans += s * factor[i];
        for (int x=a[i]+1; x<=n; x += x&(-x)) ++c[x];
    }
    return ans;    // 编号从0开始
}
```

当然，运用线段树也可以在 $O(n\log n)$ 复杂度下实现康托展开，不过树状数组实现起来更简洁。

康托逆展开的 $O(n\log n)$ 复杂度算法则是线段树实现比较合适：

```
#define N 10010
int a[N], c[N<<2], n, s; long long factor[N];
int query(int o, int l, int r) {
    int ans = 1;
    if (l < r) {
        int m = (l+r) >> 1, lc = o << 1, rc = lc + 1;
        if (c[o] == r-l+1) c[lc] = m-l+1, c[rc] = r-m;
        ans = c[lc] < s ? (s -= c[lc], query(rc, m+1, r)) : query(lc, l, m);
        c[o] = c[lc] + c[rc];
    } else c[o] = 0;
    return ans;
}
void decantor(long long num) {
    c[1] = n;
    --num; // 先将编号转化成0开始
    for (int i=n-1; i>=0; --i) {
        s = num / factor[i] + 1;
        a[n-1-i] = query(1, 1, n);
        num %= factor[i];
    }
}
```

康托逆展开可以通过提交U[Va11525](#)来验证，给出一份ac代码：

```
#include <iostream>
using namespace std;

int c[1<<17], k, s;

int query(int o, int l, int r) {
    int ans = 1;
    if (l < r) {
        int m = (l+r) >> 1, lc = o << 1, rc = lc + 1;
        if (c[o] == r-l+1) c[lc] = m-l+1, c[rc] = r-m;
        ans = c[lc] < s ? (s -= c[lc], query(rc, m+1, r)) : query(lc, l, m);
        c[o] = c[lc] + c[rc];
    } else c[o] = 0;
    return ans;
}

void solve() {
    cin >> k; c[1] = k;
    for (int i=1; i<=k; ++i) {
        cin >> s; ++s;
        cout << query(1, 1, k);
        i < k ? cout << ' ' : cout << endl;
    }
}

int main() {
    ios::sync_with_stdio(false); cin.tie(0); cout.tie(0);
    int t; cin >> t;
    while (t--) solve();
    return 0;
}
```